ZKPay — Income-Verified Instant Insurance Claim Settlement

Full Monorepo Technical Specification

> **Purpose of this document:** Complete, self-contained build spec for Claude Code.
> Every architectural decision is pre-made. Every credential is embedded (all are test/demo values).
> Build exactly what is described. Do not improvise architecture. Ask only for blockers.

Table of Contents

1. Product Overview
2. Monorepo Structure
3. Architecture & Data Flow
4. Credentials & Environment Variables
5. Component 1 — Noir Circuit
6. Component 2 — CosmWasm Contract
7. Component 3 — Backend API (Node.js)
8. Component 4 — Frontend (Next.js)
9. Shared Types Package
10. Integration Contracts Between Components
11. Build & Run Instructions
12. Known Stubs & Placeholders
13. Testing Checklist

---

1. Product Overview

The Problem (Real Insurer Anecdotes)

- **Workers' comp**: Insurers must call HR to verify last paycheck before claim payout — slow, manual, fraud-prone.
- **Business interruption**: People lie about income; insurers want direct source-of-truth links.
- **Friday-night displacement**: A family displaced by a fire on Friday needs money before Monday. Current systems can't do instant payout because income verification takes days.

The Solution

ZKPay is an income-verified instant claim settlement system:

1. Claimant enters wallet address and claim amount into a web form.
2. Backend fetches their Gusto payroll data via zkFetch (Reclaim Protocol's zkTLS).
3. A Noir circuit proves income eligibility (salary ≥ threshold AND claim ≤ $5,000 ceiling) **without revealing the actual salary**.
4. The Noir proof is submitted to a CosmWasm contract on the Xion blockchain.
5. The contract calls the on-chain Barretenberg verifier module, checks the proof, and if valid, initiates a stablecoin payout.
6. The claimant can query their claim status asynchronously from the frontend.

Key Design Decisions (Pre-made, Do Not Change)

Decision	Choice	Reason
Payroll data source	Gusto Demo API	Has real pay stub endpoints with gross_pay
zkTLS / proof of fetch	Reclaim Protocol zkFetch SDK	No dashboard needed; works with raw URL + headers
Privacy layer	Noir circuit + Barretenberg	Proves threshold without revealing salary
Blockchain	Xion (Burnt Labs testnet)	Has deployed Barretenberg verifier module
On-chain proof verifier	Barretenberg verifier module (existing)	Already deployed, accepts UltraPlonk proofs
Claim payout	CosmWasm contract → UXION token transfer	Placeholder treasury address for demo
Frontend	Next.js (web)	Best for hackathon demo; judges can see it in browser
Backend	Node.js + Express	zkFetch SDK is Node-native; proof generation runs server-side
Monorepo tooling	pnpm workspaces	Fast, simple, works across packages

---

2. Monorepo Structure

```
zkpay/
├── package.json              # pnpm workspace root
├── pnpm-workspace.yaml
├── .env.example              # All env vars documented
├── .gitignore
│
```

```
├── packages/
│   └── types/                  # Shared TypeScript types
│       ├── package.json
│       └── src/
│           └── index.ts        # ClaimRequest, ProofResult, ClaimStatus, etc.
│
├── circuit/                    # Noir ZK circuit
│   ├── Nargo.toml
│   └── src/
│       └── main.nr             # Income threshold circuit
│
├── contracts/                  # CosmWasm smart contract (Rust)
│   ├── Cargo.toml
│   └── src/
│       ├── lib.rs
│       ├── msg.rs              # InstantiateMsg, ExecuteMsg, QueryMsg
│       ├── state.rs            # Claim storage
│       ├── contract.rs         # Main entry points
│       └── error.rs
│
├── backend/                    # Node.js Express API
│   ├── package.json
│   ├── tsconfig.json
│   └── src/
│       ├── index.ts            # Express app entry
│       ├── routes/
│       │   ├── claim.ts        # POST /api/claim, GET /api/claim/:id
│       │   └── health.ts
│       ├── services/
│       │   ├── zkfetch.ts      # Reclaim zkFetch integration
│       │   ├── noir-prover.ts  # Noir proof generation via nargo
│       │   └── xion-client.ts  # Cosmos SDK client, tx submission
│       └── config.ts           # All env vars loaded here
│
└── frontend/                   # Next.js web application
    ├── package.json
    ├── tsconfig.json
    ├── next.config.ts
    └── src/
        ├── app/
        │   ├── layout.tsx
        │   ├── page.tsx        # Landing / claim form
        │   └── claim/
        │       └── [id]/
        │           └── page.tsx  # Claim status page
        ├── components/
        │   ├── ClaimForm.tsx     # Main form UI
```

```
│   ├── ClaimStatus.tsx     # Async status checker
│   ├── ProofProgress.tsx   # Step-by-step progress indicator
│   └── ui/                 # Shared UI primitives
├── hooks/
│   └── useClaim.ts         # Claim submission + polling logic
└── lib/
    └── api.ts              # Typed API client (calls backend)
```

---

3. Architecture & Data Flow

End-to-End Flow

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          FRONTEND (Next.js)                              │
│                                                                          │
│  ClaimForm                                                               │
│  ┌──────────────────────────────────────────────────────────────┐        │
│  │ Wallet Address: [xion1abc...]                                │        │
│  │ Claim Amount:   [$2,500]                                     │        │
│  │ Claim Reason:   [House Fire ▼]                               │        │
│  │                                                              │        │
│  │  [Submit Claim]                                              │        │
│  └──────────────────────────────────────────────────────────────┘        │
│                                                                          │
│                                  │                                       │
│                                  │ POST /api/claim                       │
└──────────────────────────────────┼───────────────────────────────────────┘
                                   │
                                   ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                       BACKEND (Node.js / Express)                        │
│                                                                          │
│  1. Receive claim request                                                │
│     { walletAddress, claimAmount, employeeUuid }                         │
│                                                                          │
│  2. zkFetch: Call Gusto API with private Bearer token                    │
│     GET https://api.gusto-demo.com/v1/employees/{uuid}/pay_stubs         │
│     → Reclaim witness network attests HTTPS response                     │
│     → Returns proof + extractedParameters { grossPay, checkDate }        │
│                                                                          │
│  3. Verify Reclaim attestation signatures (verifyProof)                  │
│                                                                          │
│  4. Extract grossPay from proof context                                  │
│     grossPay = parseFloat(extractedParameters.grossPay) * 100            │
│     (convert to cents as u64 for Noir)                                   │
│                                                                          │
```

```
│ 5. Generate Noir proof                                          │
│    Write Prover.toml with { gross_pay, check_date_epoch,        │
│      claim_amount, income_threshold, max_auto_payout,           │
│      max_pay_stub_age }                                         │
│    Execute: nargo prove                                         │
│    Read: ./proofs/main.proof + ./Verifier.toml (public inputs)  │
│                                                                 │
│ 6. Submit to Xion                                               │
│    MsgExecuteContract → verifier contract                       │
│    { SubmitClaim: { noir_proof, public_inputs, reclaim_proof,   │
│                     wallet_address, claim_amount } }            │
│                                                                 │
│ 7. Store claim state locally (in-memory map, keyed by claim_id) │
│    Return { claimId, txHash, status: "submitted" }              │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌─────────────────────────────────────────────────────────────────┐
│                   XION BLOCKCHAIN (Testnet)                     │
│                                                                 │
│ CosmWasm Contract: zkpay-verifier                               │
│ ┌─────────────────────────────────────────────────────────────┐ │
│ │ ExecuteMsg::SubmitClaim                                     │ │
│ │                                                             │ │
│ │ 1. Validate claim_amount <= MAX_AUTO_PAYOUT (500000)        │ │
│ │    (amounts in uUXION, $5000 = 500000 units)                │ │
│ │                                                             │ │
│ │ 2. Query Barretenberg verifier module (wasm querier)        │ │
│ │    deps.querier.query_wasm_smart(                           │ │
│ │      BB_VERIFIER_ADDR,                                      │ │
│ │      VerifyProof { proof, public_inputs, vkey_hash }        │ │
│ │    )                                                        │ │
│ │    → Returns { valid: true/false }                          │ │
│ │                                                             │ │
│ │ 3. [FOR NOW: always return valid=true, skip step 2]         │ │
│ │                                                             │ │
│ │ 4. If valid:                                                │ │
│ │    - Store claim: { id, wallet, amount, status:             │ │
│ │      "approved", timestamp }                                │ │
│ │    - Emit BankMsg::Send from TREASURY_ADDR                  │ │
│ │      to wallet_address for claim_amount uxion               │ │
│ │                                                             │ │
│ │ 5. If invalid:                                              │ │
│ │    - Store claim: { status: "rejected" }                    │ │
│ │                                                             │ │
│ └─────────────────────────────────────────────────────────────┘ │
│                                                                 │
```

```
│  QueryMsg::GetClaim { id } → ClaimStatus                    │
│  QueryMsg::ListClaims { wallet } → Vec<ClaimStatus>         │
│                                                             │
└─────────────────────────────────────────────────────────────
```

Status Polling Flow (Async)

```
Frontend            Backend             Xion
   │                   │                  │
   │ GET /api/claim/:id │                  │
   │ ─────────────────────────────────────► │                  │
   │                   │ QueryMsg::GetClaim │
   │                   │ ─────────────────────────────────► │
   │                   │ ◄──────────────────────────────────│
   │                   │ { status, tx_hash, amt } │
   │ ◄──────────────────────────────────────│                  │
   │ { status, txHash } │                  │
```

---

4. Credentials & Environment Variables

Root `.env` (copy to each package that needs it, or use workspace root)

```bash
```

```
# ==============================================
# GUSTO DEMO API
# ==============================================
GUSTO_API_BASE_URL=https://api.gusto-demo.com
GUSTO_API_VERSION=2025-11-15
GUSTO_BEARER_TOKEN=lasKDaWB7tJucPAi2lPkxb9oUrjsBRLxbFIjbDH8V2g
GUSTO_EMPLOYEE_UUID=c3dd05c3-86aa-43e9-8f2d-c8832f5c5ae8

# ==============================================
# RECLAIM PROTOCOL (zkFetch)
# Get from: https://dev.reclaimprotocol.org
# Create a "zkFetch" application type
# Enable zkFetch in the Integration tab
# ==============================================
RECLAIM_APP_ID=YOUR_RECLAIM_APP_ID_HERE
RECLAIM_APP_SECRET=YOUR_RECLAIM_APP_SECRET_HERE

# ==============================================
# XION TESTNET
# ==============================================
XION_RPC_URL=https://rpc.xion-testnet-2.burnt.com:443
XION_REST_URL=https://api.xion-testnet-2.burnt.com
XION_CHAIN_ID=xion-testnet-2

# Backend wallet (submits transactions on behalf of users)
# This is a demo key — never use in production
XION_BACKEND_MNEMONIC=test test test test test test test test test test test junk
XION_BACKEND_ADDRESS=xion1qyqszqgpqyqszqgpqyqszqgpqyqszqgpqyqs

# Contract addresses (fill in after deployment)
ZKPAY_CONTRACT_ADDRESS=PLACEHOLDER_DEPLOY_FIRST
BB_VERIFIER_MODULE_ADDRESS=xion1barretenbergverifieraddresshere

# Treasury address for payouts (placeholder — fund this address on testnet)
TREASURY_ADDRESS=xion1treasuryplaceholderaddress000000000

# ==============================================
# CIRCUIT
# ==============================================
NARGO_WORKSPACE_PATH=../circuit
VKEY_HASH=PLACEHOLDER_COMPUTE_AFTER_COMPILE

# ==============================================
# CLAIM PARAMETERS
# All values in CENTS (u64)
# ==============================================
```

```
INCOME_THRESHOLD_CENTS=300000      # $3,000 minimum gross pay per period
MAX_AUTO_PAYOUT_CENTS=500000       # $5,000 maximum claim amount
MAX_PAY_STUB_AGE_DAYS=90           # Pay stub must be < 90 days old

# ==========================================
# BACKEND SERVER
# ==========================================
PORT=3001
FRONTEND_URL=http://localhost:3000
```

---

5. Component 1 — Noir Circuit

Location: `circuit/`

File: `circuit/Nargo.toml`

```toml
[package]
name = "income_verifier"
type = "bin"
authors = ["Burnt Labs"]
compiler_version = ">=0.38.0"

[dependencies]
```

File: `circuit/src/main.nr`

```rust
```

```
// Income Threshold Verifier
//
// Proves that a claimant's gross pay is above a minimum threshold
// and that the claim amount is within the auto-payout ceiling,
// WITHOUT revealing the actual gross pay amount.
//
// Private inputs (hidden from verifier/chain):
//   gross_pay_cents: u64   - Raw gross pay in cents (e.g., 1000000 = $10,000)
//
// Public inputs (visible on chain):
//   check_date_epoch: u64  - Unix timestamp of the pay stub date
//   claim_amount_cents: u64 - Amount the claimant is requesting, in cents
//   income_threshold_cents: u64 - Minimum gross pay required, in cents
//   max_auto_payout_cents: u64  - Maximum auto-payout ceiling, in cents
//   max_pay_stub_age_secs: u64  - Max allowed age of pay stub (unix timestamp lower bound)

fn main(
    gross_pay_cents: u64,
    check_date_epoch: pub u64,
    claim_amount_cents: pub u64,
    income_threshold_cents: pub u64,
    max_auto_payout_cents: pub u64,
    max_pay_stub_age_secs: pub u64,
) {
    // Constraint 1: Income is above the minimum threshold
    // Proves salary qualification without revealing exact income
    assert(gross_pay_cents >= income_threshold_cents);

    // Constraint 2: Claim is within the auto-payout ceiling
    // Prevents abuse of auto-approval for large claims
    assert(claim_amount_cents <= max_auto_payout_cents);

    // Constraint 3: Pay stub is recent enough to be valid
    // Prevents use of stale income data
    // check_date_epoch must be >= max_pay_stub_age_secs (a unix timestamp floor)
    assert(check_date_epoch >= max_pay_stub_age_secs);
}
```

Building the Circuit

```bash
```

```
cd circuit

# Compile circuit and generate verification key
nargo compile

# Run test proof (with sample values from Prover.toml)
nargo prove

# Verify locally
nargo verify

# After compilation, compute vkey hash for contract config:
# The vkey is in: circuit/target/income_verifier.vk
# Hash it and store in VKEY_HASH env var
```

File: `circuit/Prover.toml` (for local testing only, do not commit real values)

```toml
# Private inputs
gross_pay_cents = "1000000"      # $10,000 in cents

# Public inputs
check_date_epoch = "1741046400"   # 2026-03-04 as unix timestamp
claim_amount_cents = "250000"     # $2,500 claim
income_threshold_cents = "300000" # $3,000 minimum threshold
max_auto_payout_cents = "500000"  # $5,000 ceiling
max_pay_stub_age_secs = "1733097600" # 90 days ago from 2026-03-04
```

Expected Behavior

- With the sample values: proof **succeeds** ($10,000 > $3,000, $2,500 < $5,000, date is recent)
- Change `gross_pay_cents = "200000"` ($2,000): proof **fails** at constraint 1
- Change `claim_amount_cents = "600000"` ($6,000): proof **fails** at constraint 2

---

6. Component 2 — CosmWasm Contract

Location: `contracts/`

File: `contracts/Cargo.toml`

```toml
```

```toml
[package]
name = "zkpay-verifier"
version = "0.1.0"
edition = "2021"

[lib]
crate-type = ["cdylib", "rlib"]

[features]
backtraces = ["cosmwasm-std/backtraces"]
library = []

[dependencies]
cosmwasm-std = { version = "2.1", features = ["cosmwasm_2_0"] }
cosmwasm-schema = "2.1"
cw-storage-plus = "2.0"
schemars = "0.8"
serde = { version = "1.0", features = ["derive"] }
thiserror = "1.0"
hex = "0.4"
sha2 = "0.10"

[dev-dependencies]
cosmwasm-vm = "2.1"
```

File: `contracts/src/msg.rs`

```rust
```

```rust
use cosmwasm_schema::{cw_serde, QueryResponses};
use cosmwasm_std::Uint128;

#[cw_serde]
pub struct InstantiateMsg {
    /// Address of the Barretenberg verifier module
    pub bb_verifier_address: String,
    /// Treasury address that funds will be sent FROM
    pub treasury_address: String,
    /// Hash of the verification key (hex encoded)
    pub vkey_hash: String,
    /// Minimum income threshold in cents
    pub income_threshold_cents: u64,
    /// Maximum auto-payout in uUXION
    pub max_auto_payout: Uint128,
}

#[cw_serde]
pub enum ExecuteMsg {
    SubmitClaim {
        /// Hex-encoded Noir UltraPlonk proof bytes
        noir_proof: String,
        /// Public inputs matching circuit: [check_date_epoch, claim_amount_cents,
        ///  income_threshold_cents, max_auto_payout_cents, max_pay_stub_age_secs]
        /// Each as decimal string
        public_inputs: Vec<String>,
        /// Serialized Reclaim proof (JSON string) — stored for audit trail
        reclaim_proof_json: String,
        /// Claimant's Xion wallet address to receive payout
        wallet_address: String,
        /// Claim amount in uUXION
        claim_amount: Uint128,
        /// Human-readable claim ID from backend (UUID)
        claim_id: String,
    },
}

#[cw_serde]
#[derive(QueryResponses)]
pub enum QueryMsg {
    #[returns(ClaimStatusResponse)]
    GetClaim { id: String },

    #[returns(ListClaimsResponse)]
    ListClaims {
        wallet: String,
```

```rust
        start_after: Option<String>,
        limit: Option<u32>,
    },

    #[returns(ConfigResponse)]
    GetConfig {},
}

#[cw_serde]
pub struct ClaimStatusResponse {
    pub id: String,
    pub wallet_address: String,
    pub claim_amount: Uint128,
    pub status: ClaimStatus,
    pub tx_hash: Option<String>,
    pub timestamp: u64,
    pub rejection_reason: Option<String>,
}

#[cw_serde]
pub enum ClaimStatus {
    Pending,
    Approved,
    Rejected,
}

#[cw_serde]
pub struct ListClaimsResponse {
    pub claims: Vec<ClaimStatusResponse>,
}

#[cw_serde]
pub struct ConfigResponse {
    pub bb_verifier_address: String,
    pub treasury_address: String,
    pub vkey_hash: String,
    pub income_threshold_cents: u64,
    pub max_auto_payout: Uint128,
}

/// Query message sent to the Barretenberg verifier module
#[cw_serde]
pub struct BBVerifyQuery {
    pub proof: String,       // hex-encoded proof bytes
    pub public_inputs: Vec<String>,
    pub vkey_hash: String,   // hex-encoded vkey hash
}
```

```rust
#[cw_serde]
pub struct BBVerifyResponse {
    pub valid: bool,
}
```

File: `contracts/src/state.rs`

```rust
use cosmwasm_std::Uint128;
use cw_storage_plus::{Item, Map};
use serde::{Deserialize, Serialize};

use crate::msg::ClaimStatus;

#[derive(Serialize, Deserialize, Clone, Debug)]
pub struct Config {
    pub bb_verifier_address: String,
    pub treasury_address: String,
    pub vkey_hash: String,
    pub income_threshold_cents: u64,
    pub max_auto_payout: Uint128,
}

#[derive(Serialize, Deserialize, Clone, Debug)]
pub struct Claim {
    pub id: String,
    pub wallet_address: String,
    pub claim_amount: Uint128,
    pub status: ClaimStatus,
    pub timestamp: u64,
    pub rejection_reason: Option<String>,
}

pub const CONFIG: Item<Config> = Item::new("config");

/// Primary index: claim_id -> Claim
pub const CLAIMS: Map<&str, Claim> = Map::new("claims");

/// Secondary index: wallet_address -> Vec<claim_id>
/// Stored as a space-separated string for simplicity
pub const WALLET_CLAIMS: Map<&str, Vec<String>> = Map::new("wallet_claims");
```

File: contracts/src/contract.rs

```rust
```

```rust
use cosmwasm_std::{
    entry_point, to_json_binary, BankMsg, Binary, Coin, Deps, DepsMut,
    Env, MessageInfo, Response, StdResult, Uint128, WasmMsg, from_json,
};

use crate::error::ContractError;
use crate::msg::{
    BBVerifyQuery, BBVerifyResponse, ClaimStatus, ClaimStatusResponse,
    ConfigResponse, ExecuteMsg, InstantiateMsg, ListClaimsResponse, QueryMsg,
};
use crate::state::{Claim, Config, CLAIMS, CONFIG, WALLET_CLAIMS};

#[entry_point]
pub fn instantiate(
    deps: DepsMut,
    _env: Env,
    _info: MessageInfo,
    msg: InstantiateMsg,
) -> Result<Response, ContractError> {
    let config = Config {
        bb_verifier_address: msg.bb_verifier_address,
        treasury_address: msg.treasury_address,
        vkey_hash: msg.vkey_hash,
        income_threshold_cents: msg.income_threshold_cents,
        max_auto_payout: msg.max_auto_payout,
    };
    CONFIG.save(deps.storage, &config)?;
    Ok(Response::new().add_attribute("action", "instantiate"))
}

#[entry_point]
pub fn execute(
    deps: DepsMut,
    env: Env,
    _info: MessageInfo,
    msg: ExecuteMsg,
) -> Result<Response, ContractError> {
    match msg {
        ExecuteMsg::SubmitClaim {
            noir_proof,
            public_inputs,
            reclaim_proof_json,
            wallet_address,
            claim_amount,
            claim_id,
        } => execute_submit_claim(
```

```
            deps,
            env,
            noir_proof,
            public_inputs,
            reclaim_proof_json,
            wallet_address,
            claim_amount,
            claim_id,
        ),
    }
}

pub fn execute_submit_claim(
    deps: DepsMut,
    env: Env,
    noir_proof: String,
    public_inputs: Vec<String>,
    _reclaim_proof_json: String,
    wallet_address: String,
    claim_amount: Uint128,
    claim_id: String,
) -> Result<Response, ContractError> {
    let config = CONFIG.load(deps.storage)?;

    // Guard: claim must be within auto-payout ceiling
    if claim_amount > config.max_auto_payout {
        return Err(ContractError::ClaimExceedsCeiling {
            amount: claim_amount,
            ceiling: config.max_auto_payout,
        });
    }

    // Guard: don't process duplicate claim IDs
    if CLAIMS.has(deps.storage, &claim_id) {
        return Err(ContractError::DuplicateClaim { id: claim_id });
    }

    // ==========================================================
    // PROOF VERIFICATION
    //
    // CURRENT STATUS: STUB — always returns valid=true
    //
    // Production implementation:
    //   let verify_query = BBVerifyQuery {
    //       proof: noir_proof.clone(),
    //       public_inputs: public_inputs.clone(),
    //       vkey_hash: config.vkey_hash.clone(),
```

```
//   };
//   let verify_response: BBVerifyResponse = deps.querier.query_wasm_smart(
//       &config.bb_verifier_address,
//       &verify_query,
//   )?;
//   let proof_valid = verify_response.valid;
//
// Replace the line below with the above block once vkey is registered
// ============================================================
let proof_valid = true; // STUB: always valid for hackathon demo

let timestamp = env.block.time.seconds();

if proof_valid {
    // Store approved claim
    let claim = Claim {
        id: claim_id.clone(),
        wallet_address: wallet_address.clone(),
        claim_amount,
        status: ClaimStatus::Approved,
        timestamp,
        rejection_reason: None,
    };
    CLAIMS.save(deps.storage, &claim_id, &claim)?;

    // Update wallet index
    let mut wallet_claim_ids = WALLET_CLAIMS
        .may_load(deps.storage, &wallet_address)?
        .unwrap_or_default();
    wallet_claim_ids.push(claim_id.clone());
    WALLET_CLAIMS.save(deps.storage, &wallet_address, &wallet_claim_ids)?;

    // Initiate payout from treasury to claimant
    // NOTE: Treasury address must have sufficient UXION balance
    let payout_msg = BankMsg::Send {
        to_address: wallet_address.clone(),
        amount: vec![Coin {
            denom: "uxion".to_string(),
            amount: claim_amount,
        }],
    };

    Ok(Response::new()
        .add_message(payout_msg)
        .add_attribute("action", "claim_approved")
        .add_attribute("claim_id", claim_id)
        .add_attribute("wallet", wallet_address)
```

```
            .add_attribute("amount", claim_amount.to_string()))
    } else {
        // Store rejected claim
        let claim = Claim {
            id: claim_id.clone(),
            wallet_address: wallet_address.clone(),
            claim_amount,
            status: ClaimStatus::Rejected,
            timestamp,
            rejection_reason: Some("Proof verification failed".to_string()),
        };
        CLAIMS.save(deps.storage, &claim_id, &claim)?;

        Ok(Response::new()
            .add_attribute("action", "claim_rejected")
            .add_attribute("claim_id", claim_id)
            .add_attribute("reason", "proof_invalid"))
    }
}

#[entry_point]
pub fn query(deps: Deps, _env: Env, msg: QueryMsg) -> StdResult<Binary> {
    match msg {
        QueryMsg::GetClaim { id } => to_json_binary(&query_claim(deps, id)?),
        QueryMsg::ListClaims {
            wallet,
            start_after,
            limit,
        } => to_json_binary(&query_list_claims(deps, wallet, start_after, limit)?),
        QueryMsg::GetConfig {} => to_json_binary(&query_config(deps)?),
    }
}

fn query_claim(deps: Deps, id: String) -> StdResult<ClaimStatusResponse> {
    let claim = CLAIMS.load(deps.storage, &id)?;
    Ok(ClaimStatusResponse {
        id: claim.id,
        wallet_address: claim.wallet_address,
        claim_amount: claim.claim_amount,
        status: claim.status,
        tx_hash: None, // populated by backend from tx response
        timestamp: claim.timestamp,
        rejection_reason: claim.rejection_reason,
    })
}

fn query_list_claims(
```

```rust
    deps: Deps,
    wallet: String,
    _start_after: Option<String>,
    limit: Option<u32>,
) -> StdResult<ListClaimsResponse> {
    let limit = limit.unwrap_or(10) as usize;
    let claim_ids = WALLET_CLAIMS
        .may_load(deps.storage, &wallet)?
        .unwrap_or_default();

    let claims: Vec<ClaimStatusResponse> = claim_ids
        .iter()
        .take(limit)
        .filter_map(|id| CLAIMS.load(deps.storage, id).ok())
        .map(|c| ClaimStatusResponse {
            id: c.id,
            wallet_address: c.wallet_address,
            claim_amount: c.claim_amount,
            status: c.status,
            tx_hash: None,
            timestamp: c.timestamp,
            rejection_reason: c.rejection_reason,
        })
        .collect();

    Ok(ListClaimsResponse { claims })
}

fn query_config(deps: Deps) -> StdResult<ConfigResponse> {
    let config = CONFIG.load(deps.storage)?;
    Ok(ConfigResponse {
        bb_verifier_address: config.bb_verifier_address,
        treasury_address: config.treasury_address,
        vkey_hash: config.vkey_hash,
        income_threshold_cents: config.income_threshold_cents,
        max_auto_payout: config.max_auto_payout,
    })
}
```

File: `contracts/src/error.rs`

```rust
```

```rust
use cosmwasm_std::Uint128;
use thiserror::Error;

#[derive(Error, Debug)]
pub enum ContractError {
    #[error("{0}")]
    Std(#[from] cosmwasm_std::StdError),

    #[error("Claim amount {amount} exceeds auto-payout ceiling {ceiling}")]
    ClaimExceedsCeiling { amount: Uint128, ceiling: Uint128 },

    #[error("Duplicate claim ID: {id}")]
    DuplicateClaim { id: String },

    #[error("Unauthorized")]
    Unauthorized {},
}
```

File: `contracts/src/lib.rs`

```rust
pub mod contract;
pub mod error;
pub mod msg;
pub mod state;
```

Contract Deployment (run after building)

```bash
```

```
cd contracts

# Build optimized wasm
docker run --rm -v "$(pwd)":/code \
  --mount type=volume,source="$(basename $(pwd))_cache",target=/target \
  --mount type=volume,source=registry_cache,target=/usr/local/cargo/registry \
  cosmwasm/optimizer:0.16.0

# Deploy to Xion testnet
# Replace with actual xiond CLI or use the xion-client.ts script
xiond tx wasm store artifacts/zkpay_verifier.wasm \
  --from $XION_BACKEND_ADDRESS \
  --chain-id xion-testnet-2 \
  --node $XION_RPC_URL \
  --gas-prices 0.025uxion \
  --gas auto \
  --gas-adjustment 1.3

# Instantiate (update CODE_ID from store tx result)
xiond tx wasm instantiate $CODE_ID '{
  "bb_verifier_address": "xion1barretenbergverifieraddresshere",
  "treasury_address": "xion1treasuryplaceholderaddress000000000",
  "vkey_hash": "PLACEHOLDER_COMPUTE_AFTER_COMPILE",
  "income_threshold_cents": 300000,
  "max_auto_payout": "500000000"
}' \
  --from $XION_BACKEND_ADDRESS \
  --chain-id xion-testnet-2 \
  --node $XION_RPC_URL \
  --label "zkpay-verifier-v1" \
  --no-admin \
  --gas auto \
  --gas-adjustment 1.3

# Save the resulting contract address to ZKPAY_CONTRACT_ADDRESS env var
```

---

7. Component 3 — Backend API (Node.js)

Location: `backend/`

File: `backend/package.json`

```
json
```

```
{
  "name": "@zkpay/backend",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "ts-node-dev --respawn src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js"
  },
  "dependencies": {
    "@cosmjs/cosmwasm-stargate": "^0.32.4",
    "@cosmjs/stargate": "^0.32.4",
    "@cosmjs/crypto": "^0.32.4",
    "@reclaimprotocol/zk-fetch": "^0.2.0",
    "@reclaimprotocol/js-sdk": "^2.0.0",
    "cors": "^2.8.5",
    "dotenv": "^16.4.0",
    "express": "^4.19.0",
    "uuid": "^9.0.0",
    "zod": "^3.22.0"
  },
  "devDependencies": {
    "@types/cors": "^2.8.17",
    "@types/express": "^4.17.21",
    "@types/node": "^20.0.0",
    "@types/uuid": "^9.0.8",
    "ts-node-dev": "^2.0.0",
    "typescript": "^5.0.0"
  }
}
```

File: `backend/src/config.ts`

```typescript
```

```typescript
import dotenv from 'dotenv';
import path from 'path';

dotenv.config({ path: path.resolve(__dirname, '../../.env') });

export const config = {
  port: parseInt(process.env.PORT || '3001'),
  frontendUrl: process.env.FRONTEND_URL || 'http://localhost:3000',

  gusto: {
    baseUrl: process.env.GUSTO_API_BASE_URL!,
    apiVersion: process.env.GUSTO_API_VERSION!,
    bearerToken: process.env.GUSTO_BEARER_TOKEN!,
    employeeUuid: process.env.GUSTO_EMPLOYEE_UUID!,
  },

  reclaim: {
    appId: process.env.RECLAIM_APP_ID!,
    appSecret: process.env.RECLAIM_APP_SECRET!,
  },

  xion: {
    rpcUrl: process.env.XION_RPC_URL!,
    chainId: process.env.XION_CHAIN_ID!,
    backendMnemonic: process.env.XION_BACKEND_MNEMONIC!,
    contractAddress: process.env.ZKPAY_CONTRACT_ADDRESS!,
  },

  claim: {
    incomeThresholdCents: parseInt(process.env.INCOME_THRESHOLD_CENTS || '300000'),
    maxAutoPayoutCents: parseInt(process.env.MAX_AUTO_PAYOUT_CENTS || '500000'),
    maxPayStubAgeDays: parseInt(process.env.MAX_PAY_STUB_AGE_DAYS || '90'),
  },

  circuit: {
    nargoWorkspacePath: process.env.NARGO_WORKSPACE_PATH || '../circuit',
    vkeyHash: process.env.VKEY_HASH || 'placeholder',
  },
} as const;
```

File: `backend/src/services/zkfetch.ts`

```typescript
```

```
import { ReclaimClient } from '@reclaimprotocol/zk-fetch';
import { verifyProof } from '@reclaimprotocol/js-sdk';
import { config } from '../config';

const client = new ReclaimClient(
  config.reclaim.appId,
  config.reclaim.appSecret,
);

export interface PayStubProofResult {
  grossPayCents: number;
  checkDateEpoch: number;
  rawProof: unknown;
  proofJson: string;
}

export async function fetchPayStubProof(
  employeeUuid: string = config.gusto.employeeUuid,
): Promise<PayStubProofResult> {
  const url = `${config.gusto.baseUrl}/v1/employees/${employeeUuid}/pay_stubs`;

  const proof = await client.zkFetch(
    url,

    // publicOptions — visible in proof output
    {
      method: 'GET',
      headers: {
        'accept': 'application/json',
        'X-Gusto-API-Version': config.gusto.apiVersion,
      },
      // Match confirms response contains a valid non-zero gross_pay
      // Named capture group 'grossPay' is extracted into extractedParameters
      responseMatches: [
        {
          type: 'regex' as const,
          value: '"gross_pay":"(?<grossPay>[1-9][0-9]*\\.[0-9]+)"',
        },
        {
          type: 'regex' as const,
          value: '"check_date":"(?<checkDate>[0-9]{4}-[0-9]{2}-[0-9]{2})"',
        },
      ],
    },

    // privateOptions — hidden from verifier
```

```
  {
    headers: {
      // Bearer token is NEVER revealed in the proof
      'authorization': `Bearer ${config.gusto.bearerToken}`,
    },
    // Redact gross_pay from the proof body — the match above confirms
    // it exists and is non-zero; the value is stripped from proof output
    responseRedactions: [
      {
        regex: '"gross_pay":"(?<grossPay>[1-9][0-9]*\\.[0-9]+)"',
      },
    ],
  },

  3,    // retries
  5000, // retryInterval ms
);

// Verify Reclaim attestation signatures
const isValid = await verifyProof(proof as any);
if (!isValid) {
  throw new Error('Reclaim proof verification failed — attestation signatures invalid');
}

// Extract parameters from proof context
const context = JSON.parse((proof as any).claimInfo.context);
const { grossPay, checkDate } = context.extractedParameters;

if (!grossPay) {
  throw new Error(
    'grossPay not found in extractedParameters. ' +
    'Check if responseRedactions stripped the field. ' +
    'If so, read gross_pay directly from proof context without redaction.',
  );
}

// Convert to cents (u64 compatible)
const grossPayCents = Math.round(parseFloat(grossPay) * 100);

// Convert ISO date to unix timestamp
const checkDateEpoch = Math.floor(new Date(checkDate).getTime() / 1000);

return {
  grossPayCents,
  checkDateEpoch,
  rawProof: proof,
  proofJson: JSON.stringify(proof),
```

```
  };
}
```

File: `backend/src/services/noir-prover.ts`

```typescript
```

```
import { execSync } from 'child_process';
import fs from 'fs';
import path from 'path';
import { config } from '../config';

export interface NoirProofResult {
  proofHex: string;
  publicInputs: string[];
}

export interface CircuitInputs {
  grossPayCents: number;
  checkDateEpoch: number;
  claimAmountCents: number;
  incomeThresholdCents: number;
  maxAutoPayoutCents: number;
  maxPayStubAgeSecs: number;
}

function computeMaxPayStubAgeSecs(maxAgeDays: number): number {
  const nowSecs = Math.floor(Date.now() / 1000);
  return nowSecs - maxAgeDays * 24 * 60 * 60;
}

export async function generateNoirProof(
  inputs: CircuitInputs,
): Promise<NoirProofResult> {
  const circuitPath = path.resolve(__dirname, '../../..', config.circuit.nargoWorkspacePath);

  // Write Prover.toml with circuit inputs
  // Private inputs come first, then public inputs
  const proverToml = `# Auto-generated by zkpay backend — do not commit
# Private inputs
gross_pay_cents = "${inputs.grossPayCents}"

# Public inputs
check_date_epoch = "${inputs.checkDateEpoch}"
claim_amount_cents = "${inputs.claimAmountCents}"
income_threshold_cents = "${inputs.incomeThresholdCents}"
max_auto_payout_cents = "${inputs.maxAutoPayoutCents}"
max_pay_stub_age_secs = "${inputs.maxPayStubAgeSecs}"
`;

  const proverTomlPath = path.join(circuitPath, 'Prover.toml');
  fs.writeFileSync(proverTomlPath, proverToml);
```

```
  try {
    // Run nargo prove
    execSync('nargo prove', {
      cwd: circuitPath,
      stdio: 'pipe', // suppress output
      timeout: 120_000, // 2 minute timeout
    });
  } catch (err: any) {
    const stderr = err.stderr?.toString() || '';
    throw new Error(`nargo prove failed: ${stderr}`);
  }

  // Read proof output
  const proofPath = path.join(circuitPath, 'proofs', 'main.proof');
  if (!fs.existsSync(proofPath)) {
    throw new Error(`Proof file not found at ${proofPath}. Did nargo prove succeed?`);
  }
  const proofHex = fs.readFileSync(proofPath, 'utf8').trim();

  // Public inputs are in Verifier.toml after nargo prove
  const verifierTomlPath = path.join(circuitPath, 'Verifier.toml');
  const verifierTomlContent = fs.readFileSync(verifierTomlPath, 'utf8');

  // Parse Verifier.toml to extract public input values in order
  // Order must match circuit definition: check_date_epoch, claim_amount_cents,
  //   income_threshold_cents, max_auto_payout_cents, max_pay_stub_age_secs
  const publicInputs = [
    inputs.checkDateEpoch.toString(),
    inputs.claimAmountCents.toString(),
    inputs.incomeThresholdCents.toString(),
    inputs.maxAutoPayoutCents.toString(),
    inputs.maxPayStubAgeSecs.toString(),
  ];

  return { proofHex, publicInputs };
}

export function buildCircuitInputs(
  grossPayCents: number,
  checkDateEpoch: number,
  claimAmountCents: number,
): CircuitInputs {
  return {
    grossPayCents,
    checkDateEpoch,
    claimAmountCents,
    incomeThresholdCents: config.claim.incomeThresholdCents,
```

```
    maxAutoPayoutCents: config.claim.maxAutoPayoutCents,
    maxPayStubAgeSecs: computeMaxPayStubAgeSecs(config.claim.maxPayStubAgeDays),
  };
}
```

File: `backend/src/services/xion-client.ts`

```typescript
```

```
import { SigningCosmWasmClient } from '@cosmjs/cosmwasm-stargate';
import { DirectSecp256k1HdWallet } from '@cosmjs/crypto';
import { GasPrice } from '@cosmjs/stargate';
import { config } from '../config';

let _client: SigningCosmWasmClient | null = null;

async function getClient(): Promise<SigningCosmWasmClient> {
  if (_client) return _client;

  const wallet = await DirectSecp256k1HdWallet.fromMnemonic(
    config.xion.backendMnemonic,
    { prefix: 'xion' },
  );

  _client = await SigningCosmWasmClient.connectWithSigner(
    config.xion.rpcUrl,
    wallet,
    { gasPrice: GasPrice.fromString('0.025uxion') },
  );

  return _client;
}

export interface SubmitClaimOnChainParams {
  noirProofHex: string;
  publicInputs: string[];
  reclaimProofJson: string;
  walletAddress: string;
  claimAmountUxion: string; // amount in uxion (micro)
  claimId: string;
}

export interface OnChainSubmitResult {
  txHash: string;
  height: number;
  gasUsed: number;
}

export async function submitClaimOnChain(
  params: SubmitClaimOnChainParams,
): Promise<OnChainSubmitResult> {
  const client = await getClient();

  const wallet = await DirectSecp256k1HdWallet.fromMnemonic(
    config.xion.backendMnemonic,
```

```
    { prefix: 'xion' },
  );
  const [account] = await wallet.getAccounts();

  const msg = {
    submit_claim: {
      noir_proof: params.noirProofHex,
      public_inputs: params.publicInputs,
      reclaim_proof_json: params.reclaimProofJson,
      wallet_address: params.walletAddress,
      claim_amount: params.claimAmountUxion,
      claim_id: params.claimId,
    },
  };

  const result = await client.execute(
    account.address,
    config.xion.contractAddress,
    msg,
    'auto',
    `ZKPay claim ${params.claimId}`,
  );

  return {
    txHash: result.transactionHash,
    height: result.height,
    gasUsed: result.gasUsed,
  };
}

export async function queryClaimStatus(claimId: string) {
  const client = await getClient();

  const result = await client.queryContractSmart(config.xion.contractAddress, {
    get_claim: { id: claimId },
  });

  return result;
}

export async function queryClaimsByWallet(walletAddress: string) {
  const client = await getClient();

  const result = await client.queryContractSmart(config.xion.contractAddress, {
    list_claims: { wallet: walletAddress, limit: 20 },
  });
```

```
  return result;
}
```

File: `backend/src/routes/claim.ts`

```typescript
```

```typescript
import { Router, Request, Response } from 'express';
import { z } from 'zod';
import { v4 as uuidv4 } from 'uuid';
import { fetchPayStubProof } from '../services/zkfetch';
import { generateNoirProof, buildCircuitInputs } from '../services/noir-prover';
import { submitClaimOnChain, queryClaimStatus, queryClaimsByWallet } from '../services/xion-client';
import { config } from '../config';

const router = Router();

// In-memory claim store (for demo — maps claim_id to status for fast reads)
const claimStore = new Map<string, {
  id: string;
  status: 'processing' | 'submitted' | 'approved' | 'rejected' | 'error';
  txHash?: string;
  error?: string;
  walletAddress: string;
  claimAmountCents: number;
  createdAt: number;
}>();

const ClaimRequestSchema = z.object({
  walletAddress: z.string().startsWith('xion1').min(10),
  claimAmountDollars: z.number().positive().max(5000),
  claimReason: z.enum([
    'house_fire',
    'flood_damage',
    'theft',
    'vehicle_accident',
    'medical_emergency',
    'other',
  ]),
  employeeUuid: z.string().uuid().optional(), // defaults to demo employee
});

// POST /api/claim — submit a new insurance claim
router.post('/', async (req: Request, res: Response) => {
  const parseResult = ClaimRequestSchema.safeParse(req.body);
  if (!parseResult.success) {
    return res.status(400).json({ error: 'Invalid request', details: parseResult.error.issues });
  }

  const { walletAddress, claimAmountDollars, claimReason, employeeUuid } = parseResult.data;
  const claimAmountCents = Math.round(claimAmountDollars * 100);
  const claimId = uuidv4();
```

```
  // Reject immediately if over ceiling
  if (claimAmountCents > config.claim.maxAutoPayoutCents) {
    return res.status(400).json({
      error: `Claim amount $${claimAmountDollars} exceeds auto-payout ceiling of $${config.claim.maxAutoPayoutCents / 1
    });
  }

  // Store initial state
  claimStore.set(claimId, {
    id: claimId,
    status: 'processing',
    walletAddress,
    claimAmountCents,
    createdAt: Date.now(),
  });

  // Return claim ID immediately — processing is async
  res.status(202).json({
    claimId,
    status: 'processing',
    message: 'Claim submitted. Use GET /api/claim/:id to check status.',
  });

  // Process async (don't await)
  processClaim(claimId, walletAddress, claimAmountCents, employeeUuid).catch(
    (err) => {
      console.error(`Claim ${claimId} processing error:`, err);
      const entry = claimStore.get(claimId);
      if (entry) {
        claimStore.set(claimId, { ...entry, status: 'error', error: err.message });
      }
    },
  );
});

async function processClaim(
  claimId: string,
  walletAddress: string,
  claimAmountCents: number,
  employeeUuid?: string,
) {
  const update = (updates: Partial<typeof claimStore extends Map<any, infer V> ? V : never>) => {
    const current = claimStore.get(claimId)!;
    claimStore.set(claimId, { ...current, ...updates } as any);
  };

  // Step 1: Fetch pay stub proof via zkFetch
```

```
  console.log(`[${claimId}] Step 1: Fetching pay stub proof via zkFetch...`);
  const payStubProof = await fetchPayStubProof(employeeUuid);
  console.log(`[${claimId}] Step 1 done: grossPay=${payStubProof.grossPayCents} cents, checkDate=${payStubProof.check

  // Step 2: Generate Noir proof
  console.log(`[${claimId}] Step 2: Generating Noir proof...`);
  const circuitInputs = buildCircuitInputs(
    payStubProof.grossPayCents,
    payStubProof.checkDateEpoch,
    claimAmountCents,
  );
  const noirProof = await generateNoirProof(circuitInputs);
  console.log(`[${claimId}] Step 2 done: proof generated (${noirProof.proofHex.length / 2} bytes)`);

  // Step 3: Submit to Xion
  console.log(`[${claimId}] Step 3: Submitting to Xion...`);
  // Convert cents to uxion: $1 = 1,000,000 uxion in this demo
  // Adjust ratio based on actual UXION/USD rate for testnet
  const claimAmountUxion = (claimAmountCents * 10000).toString(); // placeholder conversion

  const onChainResult = await submitClaimOnChain({
    noirProofHex: noirProof.proofHex,
    publicInputs: noirProof.publicInputs,
    reclaimProofJson: payStubProof.proofJson,
    walletAddress,
    claimAmountUxion,
    claimId,
  });

  update({ status: 'submitted', txHash: onChainResult.txHash });
  console.log(`[${claimId}] Step 3 done: tx=${onChainResult.txHash}`);
}

// GET /api/claim/:id — get claim status
router.get('/:id', async (req: Request, res: Response) => {
  const { id } = req.params;
  const localState = claimStore.get(id);

  if (!localState) {
    return res.status(404).json({ error: 'Claim not found' });
  }

  // If submitted, also fetch on-chain state for confirmed status
  let onChainStatus = null;
  if (localState.status === 'submitted' || localState.status === 'approved') {
    try {
      onChainStatus = await queryClaimStatus(id);
```

```
    } catch {
      // On-chain query failed — return local state only
    }
  }

  return res.json({
    id,
    status: onChainStatus?.status ?? localState.status,
    txHash: localState.txHash,
    walletAddress: localState.walletAddress,
    claimAmountDollars: localState.claimAmountCents / 100,
    onChain: onChainStatus,
    error: localState.error,
    createdAt: localState.createdAt,
  });
});

// GET /api/claim/wallet/:address — list claims by wallet
router.get('/wallet/:address', async (req: Request, res: Response) => {
  const { address } = req.params;
  try {
    const claims = await queryClaimsByWallet(address);
    return res.json(claims);
  } catch (err: any) {
    return res.status(500).json({ error: err.message });
  }
});

export default router;
```

File: `backend/src/index.ts`

```typescript
```

```
import express from 'express';
import cors from 'cors';
import { config } from './config';
import claimRouter from './routes/claim';

const app = express();

app.use(cors({ origin: config.frontendUrl }));
app.use(express.json());

app.get('/health', (_req, res) => res.json({ status: 'ok' }));
app.use('/api/claim', claimRouter);

app.listen(config.port, () => {
  console.log(`ZKPay backend running on http://localhost:${config.port}`);
});
```

---

8. Component 4 — Frontend (Next.js)

Location: `frontend/`

File: `frontend/package.json`

```
json
```

```
{
  "name": "@zkpay/frontend",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "next": "^14.2.0",
    "react": "^18.0.0",
    "react-dom": "^18.0.0",
    "@zkpay/types": "workspace:*",
    "lucide-react": "^0.400.0",
    "clsx": "^2.1.0"
  },
  "devDependencies": {
    "@types/node": "^20.0.0",
    "@types/react": "^18.0.0",
    "@types/react-dom": "^18.0.0",
    "autoprefixer": "^10.4.0",
    "postcss": "^8.4.0",
    "tailwindcss": "^3.4.0",
    "typescript": "^5.0.0"
  }
}
```

File: `frontend/src/app/layout.tsx`

```
tsx
```

```
import type { Metadata } from 'next';
import './globals.css';

export const metadata: Metadata = {
  title: 'ZKPay — Instant Insurance Claims',
  description: 'Income-verified instant insurance claim settlement powered by zero-knowledge proofs on Xion.',
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body className="bg-gray-950 text-white antialiased">
        {children}
      </body>
    </html>
  );
}
```

File: `frontend/src/app/globals.css`

```
css

@tailwind base;
@tailwind components;
@tailwind utilities;
```

File: `frontend/src/lib/api.ts`

```
typescript
```

```typescript
const API_BASE = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:3001';

export interface SubmitClaimRequest {
  walletAddress: string;
  claimAmountDollars: number;
  claimReason: string;
  employeeUuid?: string;
}

export interface ClaimResponse {
  id: string;
  status: 'processing' | 'submitted' | 'approved' | 'rejected' | 'error';
  txHash?: string;
  walletAddress: string;
  claimAmountDollars: number;
  onChain?: {
    status: string;
    claim_amount: string;
    timestamp: number;
  };
  error?: string;
  createdAt: number;
}

export async function submitClaim(data: SubmitClaimRequest): Promise<{ claimId: string; status: string }> {
  const res = await fetch(`${API_BASE}/api/claim`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  });
  if (!res.ok) {
    const err = await res.json();
    throw new Error(err.error || 'Failed to submit claim');
  }
  return res.json();
}

export async function getClaimStatus(claimId: string): Promise<ClaimResponse> {
  const res = await fetch(`${API_BASE}/api/claim/${claimId}`);
  if (!res.ok) {
    const err = await res.json();
    throw new Error(err.error || 'Failed to fetch claim status');
  }
  return res.json();
}
```

File: frontend/src/hooks/useClaim.ts

```typescript
```

```
'use client';

import { useState, useCallback, useRef, useEffect } from 'react';
import { submitClaim, getClaimStatus, type ClaimResponse } from '../lib/api';

type ClaimState =
  | { phase: 'idle' }
  | { phase: 'submitting' }
  | { phase: 'polling'; claimId: string; status: ClaimResponse | null }
  | { phase: 'done'; result: ClaimResponse }
  | { phase: 'error'; message: string };

export function useClaim() {
  const [state, setState] = useState<ClaimState>({ phase: 'idle' });
  const pollRef = useRef<NodeJS.Timeout | null>(null);

  const stopPolling = useCallback(() => {
    if (pollRef.current) {
      clearInterval(pollRef.current);
      pollRef.current = null;
    }
  }, []);

  // Auto-stop polling when done
  useEffect(() => {
    if (state.phase === 'done' || state.phase === 'error') {
      stopPolling();
    }
  }, [state.phase, stopPolling]);

  const submit = useCallback(async (
    walletAddress: string,
    claimAmountDollars: number,
    claimReason: string,
  ) => {
    setState({ phase: 'submitting' });

    try {
      const { claimId } = await submitClaim({ walletAddress, claimAmountDollars, claimReason });
      setState({ phase: 'polling', claimId, status: null });

      // Poll every 3 seconds
      pollRef.current = setInterval(async () => {
        const status = await getClaimStatus(claimId);
        setState({ phase: 'polling', claimId, status });
```

```
      // Stop polling on terminal states
      if (['approved', 'rejected', 'error'].includes(status.status)) {
        stopPolling();
        setState({ phase: 'done', result: status });
      }
    }, 3000);
  } catch (err: any) {
    setState({ phase: 'error', message: err.message });
  }
}, [stopPolling]);

const checkStatus = useCallback(async (claimId: string) => {
  try {
    const status = await getClaimStatus(claimId);
    setState({ phase: 'done', result: status });
  } catch (err: any) {
    setState({ phase: 'error', message: err.message });
  }
}, []);

const reset = useCallback(() => {
  stopPolling();
  setState({ phase: 'idle' });
}, [stopPolling]);

return { state, submit, checkStatus, reset };
}
```

File: `frontend/src/components/ProofProgress.tsx`

```tsx
```

```
'use client';

import { CheckCircle, Circle, Loader2 } from 'lucide-react';

interface Step {
  id: string;
  label: string;
  description: string;
}

const STEPS: Step[] = [
  {
    id: 'fetch',
    label: 'Fetching Payroll Data',
    description: 'Connecting to Gusto via zkTLS — your salary stays private',
  },
  {
    id: 'prove',
    label: 'Generating ZK Proof',
    description: 'Proving income eligibility without revealing your salary',
  },
  {
    id: 'submit',
    label: 'Submitting to Xion',
    description: 'Sending verified proof to the blockchain',
  },
  {
    id: 'payout',
    label: 'Processing Payout',
    description: 'Contract verified — initiating stablecoin transfer',
  },
];

type StepStatus = 'pending' | 'active' | 'done' | 'error';

function inferStepStatus(
  appStatus: string,
  stepIndex: number,
): StepStatus {
  const statusToProgress: Record<string, number> = {
    processing: 1,
    submitted: 3,
    approved: 4,
    rejected: 4,
    error: -1,
  };
```

```
  const progress = statusToProgress[appStatus] ?? 0;
  if (progress < 0) return stepIndex === 0 ? 'error' : 'pending';
  if (stepIndex < progress) return 'done';
  if (stepIndex === progress) return 'active';
  return 'pending';
}

export function ProofProgress({ status }: { status: string }) {
  return (
    <div className="space-y-4">
      {STEPS.map((step, i) => {
        const stepStatus = inferStepStatus(status, i);
        return (
          <div key={step.id} className="flex items-start gap-4">
            <div className="flex-shrink-0 mt-0.5">
              {stepStatus === 'done' && (
                <CheckCircle className="w-5 h-5 text-emerald-400" />
              )}
              {stepStatus === 'active' && (
                <Loader2 className="w-5 h-5 text-blue-400 animate-spin" />
              )}
              {stepStatus === 'pending' && (
                <Circle className="w-5 h-5 text-gray-600" />
              )}
              {stepStatus === 'error' && (
                <Circle className="w-5 h-5 text-red-400" />
              )}
            </div>
            <div>
              <div className={`text-sm font-medium ${
                stepStatus === 'done' ? 'text-emerald-400' :
                stepStatus === 'active' ? 'text-white' :
                stepStatus === 'error' ? 'text-red-400' :
                'text-gray-500'
              }`}>
                {step.label}
              </div>
              <div className="text-xs text-gray-500 mt-0.5">{step.description}</div>
            </div>
          </div>
        );
      })}
    </div>
  );
}
```

File: frontend/src/components/ClaimForm.tsx

```tsx
```

```
'use client';

import { useState } from 'react';
import { useClaim } from '../hooks/useClaim';
import { ProofProgress } from './ProofProgress';
import { Shield, Zap, CheckCircle, XCircle, ExternalLink } from 'lucide-react';

const CLAIM_REASONS = [
  { value: 'house_fire', label: '🔥 House Fire' },
  { value: 'flood_damage', label: '🌊 Flood Damage' },
  { value: 'theft', label: '🔒 Theft' },
  { value: 'vehicle_accident', label: '🚗 Vehicle Accident' },
  { value: 'medical_emergency', label: '🏥 Medical Emergency' },
  { value: 'other', label: '📋 Other' },
];

const XION_EXPLORER_BASE = 'https://explorer.xion-testnet-2.burnt.com/xion-testnet-2/tx';

export function ClaimForm() {
  const { state, submit, checkStatus, reset } = useClaim();
  const [walletAddress, setWalletAddress] = useState('');
  const [claimAmount, setClaimAmount] = useState('');
  const [claimReason, setClaimReason] = useState('house_fire');
  const [lookupId, setLookupId] = useState('');

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    const amount = parseFloat(claimAmount);
    if (isNaN(amount) || amount <= 0 || amount > 5000) return;
    await submit(walletAddress, amount, claimReason);
  };

  // Idle state — show the form
  if (state.phase === 'idle') {
    return (
      <div className="space-y-6">
        <form onSubmit={handleSubmit} className="space-y-4">
          {/* Wallet Address */}
          <div>
            <label className="block text-sm font-medium text-gray-300 mb-1">
              Xion Wallet Address
            </label>
            <input
              type="text"
              value={walletAddress}
              onChange={(e) => setWalletAddress(e.target.value)}
```

```
    placeholder="xion1..."
    required
    className="w-full px-4 py-2.5 rounded-lg bg-gray-800 border border-gray-700 text-white placeholder-gray-500 foc
  />
</div>

{/* Claim Amount */}
<div>
  <label className="block text-sm font-medium text-gray-300 mb-1">
    Claim Amount (USD)
  </label>
  <div className="relative">
    <span className="absolute left-4 top-1/2 -translate-y-1/2 text-gray-400">$</span>
    <input
      type="number"
      value={claimAmount}
      onChange={(e) => setClaimAmount(e.target.value)}
      placeholder="0.00"
      min="1"
      max="5000"
      step="0.01"
      required
      className="w-full pl-8 pr-4 py-2.5 rounded-lg bg-gray-800 border border-gray-700 text-white placeholder-gray-50
    />
  </div>
  <p className="text-xs text-gray-500 mt-1">Maximum $5,000 for instant auto-approval</p>
</div>

{/* Claim Reason */}
<div>
  <label className="block text-sm font-medium text-gray-300 mb-1">
    Claim Reason
  </label>
  <select
    value={claimReason}
    onChange={(e) => setClaimReason(e.target.value)}
    className="w-full px-4 py-2.5 rounded-lg bg-gray-800 border border-gray-700 text-white focus:outline-none focus:
  >
    {CLAIM_REASONS.map((r) => (
      <option key={r.value} value={r.value}>{r.label}</option>
    ))}
  </select>
</div>

{/* Privacy Notice */}
<div className="rounded-lg bg-blue-950/50 border border-blue-800/50 px-4 py-3 text-xs text-blue-300">
  <div className="flex items-start gap-2">
```

```
            <Shield className="w-4 h-4 mt-0.5 flex-shrink-0 text-blue-400" />
            <p>
              Your income is verified using <strong>zero-knowledge proofs</strong>.
              Your actual salary is never revealed to the insurer or stored on-chain —
              only proof that you qualify.
            </p>
          </div>
        </div>

        <button
          type="submit"
          disabled={!walletAddress || !claimAmount}
          className="w-full py-3 rounded-lg bg-blue-600 hover:bg-blue-500 disabled:bg-gray-700 disabled:cursor-not-allowed
        >
          <Zap className="w-4 h-4" />
          Submit Instant Claim
        </button>
      </form>

      {/* Claim Status Lookup */}
      <div className="pt-4 border-t border-gray-800">
        <p className="text-xs text-gray-500 mb-2">Already have a claim ID?</p>
        <div className="flex gap-2">
          <input
            type="text"
            value={lookupId}
            onChange={(e) => setLookupId(e.target.value)}
            placeholder="Claim ID (UUID)"
            className="flex-1 px-3 py-2 rounded-lg bg-gray-800 border border-gray-700 text-white placeholder-gray-500 focus
          />
          <button
            onClick={() => lookupId && checkStatus(lookupId)}
            className="px-4 py-2 rounded-lg bg-gray-700 hover:bg-gray-600 text-white text-xs transition"
          >
            Check Status
          </button>
        </div>
      </div>
    </div>
  );
}

// Submitting state
if (state.phase === 'submitting') {
  return (
    <div className="text-center py-4">
      <div className="text-sm text-gray-400 mb-2">Initiating claim...</div>
```

```
    </div>
  );
}

// Polling state — show live progress
if (state.phase === 'polling') {
  return (
    <div className="space-y-6">
      <div>
        <div className="text-sm font-medium text-gray-300 mb-1">Claim ID</div>
        <div className="font-mono text-xs text-gray-400 break-all">{state.claimId}</div>
      </div>
      <ProofProgress status={state.status?.status ?? 'processing'} />
      <p className="text-xs text-center text-gray-500">
        Processing your claim... This may take 30–60 seconds while the ZK proof is generated.
      </p>
    </div>
  );
}

// Done state — approved or rejected
if (state.phase === 'done') {
  const { result } = state;
  const approved = result.status === 'approved';

  return (
    <div className="space-y-5">
      <div className={`rounded-xl p-5 text-center border ${
        approved
          ? 'bg-emerald-950/50 border-emerald-700/50'
          : 'bg-red-950/50 border-red-700/50'
      }`}>
        {approved ? (
          <CheckCircle className="w-12 h-12 text-emerald-400 mx-auto mb-3" />
        ) : (
          <XCircle className="w-12 h-12 text-red-400 mx-auto mb-3" />
        )}
        <div className={`text-xl font-bold mb-1 ${approved ? 'text-emerald-300' : 'text-red-300'}`}>
          {approved ? 'Claim Approved!' : 'Claim Rejected'}
        </div>
        {approved && (
          <div className="text-2xl font-bold text-white mt-2">
            ${result.claimAmountDollars.toLocaleString()}
          </div>
        )}
        <div className="text-xs text-gray-400 mt-2">
          {approved
```

```
            ? 'Payout has been initiated to your wallet'
            : result.onChain?.status ?? 'Proof verification failed'}
          </div>
        </div>

        {/* Transaction Hash */}
        {result.txHash && (
          <div className="rounded-lg bg-gray-800/50 border border-gray-700 px-4 py-3">
            <div className="text-xs text-gray-500 mb-1">Transaction</div>
            <a
              href={`${XION_EXPLORER_BASE}/${result.txHash}`}
              target="_blank"
              rel="noopener noreferrer"
              className="flex items-center gap-1.5 font-mono text-xs text-blue-400 hover:text-blue-300 break-all"
            >
              {result.txHash.slice(0, 16)}...{result.txHash.slice(-8)}
              <ExternalLink className="w-3 h-3 flex-shrink-0" />
            </a>
          </div>
        )}

        {/* Claim ID */}
        <div className="rounded-lg bg-gray-800/50 border border-gray-700 px-4 py-3">
          <div className="text-xs text-gray-500 mb-1">Claim ID</div>
          <div className="font-mono text-xs text-gray-300 break-all">{result.id}</div>
        </div>

        <button
          onClick={reset}
          className="w-full py-2.5 rounded-lg bg-gray-700 hover:bg-gray-600 text-white text-sm transition"
        >
          Submit Another Claim
        </button>
      </div>
    );
  }

  // Error state
  if (state.phase === 'error') {
    return (
      <div className="space-y-4">
        <div className="rounded-lg bg-red-950/50 border border-red-700/50 px-4 py-3">
          <div className="text-sm font-medium text-red-300 mb-1">Error</div>
          <div className="text-xs text-red-400">{state.message}</div>
        </div>
        <button
          onClick={reset}
```

```
        className="w-full py-2.5 rounded-lg bg-gray-700 hover:bg-gray-600 text-white text-sm transition"
      >
        Try Again
      </button>
    </div>
  );
}

return null;
}
```

File: **frontend/src/app/page.tsx**

```tsx
```

```
import { ClaimForm } from '../components/ClaimForm';
import { Shield, Zap, Lock } from 'lucide-react';

export default function HomePage() {
  return (
    <main className="min-h-screen bg-gray-950 flex flex-col">
      {/* Header */}
      <header className="border-b border-gray-800 px-6 py-4">
        <div className="max-w-6xl mx-auto flex items-center justify-between">
          <div className="flex items-center gap-2">
            <Shield className="w-6 h-6 text-blue-400" />
            <span className="font-bold text-white text-lg">ZKPay</span>
            <span className="text-xs text-gray-500 mt-0.5">by Burnt Labs</span>
          </div>
          <div className="flex items-center gap-2 text-xs text-gray-500">
            <span className="w-2 h-2 rounded-full bg-emerald-400 inline-block" />
            Xion Testnet
          </div>
        </div>
      </header>

      {/* Hero */}
      <section className="px-6 py-16 text-center">
        <div className="max-w-2xl mx-auto">
          <div className="inline-flex items-center gap-2 bg-blue-950/60 border border-blue-800/50 rounded-full px-4 py-1.5 te
            <Zap className="w-3 h-3" />
            Instant verified payouts in seconds
          </div>
          <h1 className="text-4xl font-bold text-white mb-4 leading-tight">
            Insurance Claims That Pay{' '}
            <span className="text-blue-400">Before Monday</span>
          </h1>
          <p className="text-gray-400 text-lg mb-4">
            Your income is verified instantly using zero-knowledge proofs.
            No paperwork. No HR calls. No waiting.
          </p>
          <div className="flex items-center justify-center gap-6 text-sm text-gray-500">
            <div className="flex items-center gap-1.5">
              <Lock className="w-4 h-4 text-gray-600" />
              Salary never revealed
            </div>
            <div className="flex items-center gap-1.5">
              <Shield className="w-4 h-4 text-gray-600" />
              On-chain proof
            </div>
            <div className="flex items-center gap-1.5">
```

```
        <Zap className="w-4 h-4 text-gray-600" />
        &lt; 60 seconds
      </div>
    </div>
  </div>
</section>

{/* Claim Form */}
<section className="px-6 pb-16 flex-1">
  <div className="max-w-md mx-auto">
    <div className="rounded-2xl bg-gray-900 border border-gray-800 p-6 shadow-2xl">
      <h2 className="text-base font-semibold text-white mb-5">
        Submit Instant Claim
      </h2>
      <ClaimForm />
    </div>
  </div>
</section>

{/* How It Works */}
<section className="px-6 py-12 bg-gray-900/50 border-t border-gray-800">
  <div className="max-w-3xl mx-auto">
    <h3 className="text-sm font-semibold text-gray-400 uppercase tracking-wider mb-8 text-center">
      How It Works
    </h3>
    <div className="grid grid-cols-1 md:grid-cols-4 gap-6">
      {[
        {
          step: '01',
          title: 'Enter wallet & amount',
          desc: 'Submit your Xion wallet and the amount you need',
        },
        {
          step: '02',
          title: 'zkTLS proof of income',
          desc: 'Your payroll is verified via Reclaim Protocol — no raw data leaves',
        },
        {
          step: '03',
          title: 'ZK circuit proves eligibility',
          desc: 'A Noir circuit proves income threshold is met, salary stays private',
        },
        {
          step: '04',
          title: 'On-chain payout',
          desc: 'Contract verifies the proof and sends UXION to your wallet',
        },
```

```
          ].map((item) => (
            <div key={item.step} className="text-center">
              <div className="text-2xl font-bold text-blue-800 mb-2">{item.step}</div>
              <div className="text-sm font-medium text-white mb-1">{item.title}</div>
              <div className="text-xs text-gray-500">{item.desc}</div>
            </div>
          ))}
        </div>
      </div>
    </section>
  </main>
);
}
```

File: **frontend/src/app/claim/[id]/page.tsx**

```
tsx
```

```
'use client';

import { useEffect, useState } from 'react';
import { getClaimStatus, type ClaimResponse } from '../../../lib/api';
import { ProofProgress } from '../../../components/ProofProgress';
import { Loader2, ArrowLeft } from 'lucide-react';
import Link from 'next/link';

export default function ClaimPage({ params }: { params: { id: string } }) {
  const [claim, setClaim] = useState<ClaimResponse | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let interval: NodeJS.Timeout;

    const fetch = async () => {
      try {
        const data = await getClaimStatus(params.id);
        setClaim(data);
        setLoading(false);

        if (['approved', 'rejected', 'error'].includes(data.status)) {
          clearInterval(interval);
        }
      } catch (e: any) {
        setError(e.message);
        setLoading(false);
      }
    };

    fetch();
    interval = setInterval(fetch, 3000);
    return () => clearInterval(interval);
  }, [params.id]);

  return (
    <main className="min-h-screen bg-gray-950 px-6 py-12">
      <div className="max-w-md mx-auto">
        <Link href="/" className="flex items-center gap-2 text-gray-500 hover:text-gray-300 text-sm mb-8">
          <ArrowLeft className="w-4 h-4" />
          Back to claims
        </Link>

        <div className="rounded-2xl bg-gray-900 border border-gray-800 p-6">
          <h1 className="text-base font-semibold text-white mb-1">Claim Status</h1>
```

```
    <p className="font-mono text-xs text-gray-500 mb-6 break-all">{params.id}</p>

    {loading && !claim && (
      <div className="flex items-center gap-2 text-gray-400">
        <Loader2 className="w-4 h-4 animate-spin" />
        <span className="text-sm">Loading...</span>
      </div>
    )}

    {error && (
      <div className="text-sm text-red-400">{error}</div>
    )}

    {claim && (
      <div className="space-y-6">
        <ProofProgress status={claim.status} />

        <div className="rounded-lg bg-gray-800/50 border border-gray-700 divide-y divide-gray-700/50 text-sm">
          <div className="px-4 py-3 flex justify-between">
            <span className="text-gray-500">Status</span>
            <span className={`font-medium capitalize ${
              claim.status === 'approved' ? 'text-emerald-400' :
              claim.status === 'rejected' ? 'text-red-400' :
              'text-yellow-400'
            }`}>{claim.status}</span>
          </div>
          <div className="px-4 py-3 flex justify-between">
            <span className="text-gray-500">Amount</span>
            <span className="text-white">${claim.claimAmountDollars.toLocaleString()}</span>
          </div>
          {claim.txHash && (
            <div className="px-4 py-3">
              <span className="text-gray-500 block mb-1">Transaction</span>
              <a
                href={`https://explorer.xion-testnet-2.burnt.com/xion-testnet-2/tx/${claim.txHash}`}
                target="_blank"
                rel="noopener noreferrer"
                className="font-mono text-xs text-blue-400 hover:text-blue-300 break-all"
              >
                {claim.txHash}
              </a>
            </div>
          )}
        </div>
      </div>
    )}
  </div>
```

```
    </div>
  </main>
  );
}
```

---

9. Shared Types Package

File: `packages/types/src/index.ts`

```typescript
```

```typescript
export type ClaimStatus = 'processing' | 'submitted' | 'approved' | 'rejected' | 'error';

export type ClaimReason =
  | 'house_fire'
  | 'flood_damage'
  | 'theft'
  | 'vehicle_accident'
  | 'medical_emergency'
  | 'other';

export interface SubmitClaimRequest {
  walletAddress: string;
  claimAmountDollars: number;
  claimReason: ClaimReason;
  employeeUuid?: string;
}

export interface SubmitClaimResponse {
  claimId: string;
  status: 'processing';
  message: string;
}

export interface ClaimStatusResponse {
  id: string;
  status: ClaimStatus;
  txHash?: string;
  walletAddress: string;
  claimAmountDollars: number;
  onChain?: {
    status: string;
    claim_amount: string;
    timestamp: number;
  };
  error?: string;
  createdAt: number;
}

// Circuit public inputs (order matters — matches Noir circuit signature)
export interface CircuitPublicInputs {
  checkDateEpoch: number;
  claimAmountCents: number;
  incomeThresholdCents: number;
  maxAutoPayoutCents: number;
```

```
  maxPayStubAgeSecs: number;
}
```

File: `packages/types/package.json`

```json
{
  "name": "@zkpay/types",
  "version": "0.1.0",
  "private": true,
  "main": "./src/index.ts",
  "types": "./src/index.ts"
}
```

---

10. Integration Contracts Between Components

Backend → Xion Contract (ExecuteMsg)

```json
{
  "submit_claim": {
    "noir_proof": "0xabcd...",
    "public_inputs": ["1741046400", "250000", "300000", "500000", "1733097600"],
    "reclaim_proof_json": "{...}",
    "wallet_address": "xion1abc...",
    "claim_amount": "250000000",
    "claim_id": "uuid-v4-here"
  }
}
```

Public inputs order is critical — must match circuit:

1. `check_date_epoch` (unix timestamp)
2. `claim_amount_cents`
3. `income_threshold_cents`
4. `max_auto_payout_cents`
5. `max_pay_stub_age_secs`

Frontend → Backend (POST /api/claim)

```json
```

```
{
  "walletAddress": "xion1abc...",
  "claimAmountDollars": 2500,
  "claimReason": "house_fire"
}
```

Backend → Frontend (GET /api/claim/:id)

```json
{
  "id": "uuid",
  "status": "approved",
  "txHash": "ABCD...",
  "walletAddress": "xion1...",
  "claimAmountDollars": 2500,
  "onChain": {
    "status": "Approved",
    "claim_amount": "250000000",
    "timestamp": 1741046400
  },
  "createdAt": 1741046000
}
```

---

11. Build & Run Instructions

Prerequisites

```bash
# Required tools
node >= 18
pnpm >= 8
nargo >= 0.38.0    # Install: curl -L https://raw.githubusercontent.com/noir-lang/noirup/main/install | bash && noirup
rust + cargo       # For contract compilation
docker             # For CosmWasm optimizer
```

Setup

```bash
```

```
# Clone / initialize repo
git init zkpay && cd zkpay

# Install dependencies
pnpm install

# Download Reclaim ZK files (required for zkFetch)
cd backend
node node_modules/@reclaimprotocol/zk-symmetric-crypto/lib/scripts/download-files
cd ..

# Copy env file
cp .env.example .env
# Fill in: RECLAIM_APP_ID, RECLAIM_APP_SECRET (from dev.reclaimprotocol.org)
# Fill in: ZKPAY_CONTRACT_ADDRESS (after contract deployment)
```

Running Locally

```bash
# Terminal 1 — Backend
cd backend && pnpm dev

# Terminal 2 — Frontend
cd frontend && pnpm dev

# Verify backend health
curl http://localhost:3001/health
```

Circuit: Build & Test

```bash
cd circuit

# Compile
nargo compile

# Generate test proof (uses Prover.toml values)
nargo prove

# Verify locally
nargo verify
# Should print: Verification successful
```

Contract: Build & Deploy

```bash
cd contracts

# Build wasm (optimized)
docker run --rm -v "$(pwd)":/code \
  --mount type=volume,source="$(basename $(pwd))_cache",target=/target \
  --mount type=volume,source=registry_cache,target=/usr/local/cargo/registry \
  cosmwasm/optimizer:0.16.0

# Deploy (requires xiond CLI configured for xion-testnet-2)
# See deployment commands in Section 6
```

---

12. Known Stubs & Placeholders

These are intentional stubs for the hackathon. Document each one clearly so they can be replaced post-demo:

Stub	Location	Replacement
`proof_valid = true`	`contracts/src/contract.rs:execute_submit_claim`	Uncomment BB verifier wa query once vkey is registere
`RECLAIM_APP_ID` / `RECLAIM_APP_SECRET`	`.env`	Create app at dev.reclaimprotocol.org
`ZKPAY_CONTRACT_ADDRESS`	`.env`	Populate after `xiond tx wa instantiate`
`VKEY_HASH`	`.env`	Compute from `circuit/target/income_veri` after `nargo compile`
`BB_VERIFIER_MODULE_ADDRESS`	`.env`	Get from Burnt Labs intern — already deployed on test
`XION_BACKEND_MNEMONIC`	`.env`	Real funded testnet wallet - Burnt faucet
`TREASURY_ADDRESS`	`.env`	Funded testnet address; con sends FROM this address
`claimAmountUxion conversion`	`backend/src/routes/claim.ts`	Actual UXION/USD conve rate; currently `cents * 100`
Gusto `employeeUuid`	`.env`	Demo hardcoded; productic needs OAuth delegated flov

Trust Model (for Demo Pitch)

> "The Reclaim proof attests that a legitimate Gusto API call was made with valid credentials and returned a real payroll response. The Noir circuit proves income eligibility over the extracted gross_pay without revealing it on-chain. The chain verifies the ZK proof.
> The trust assumption is: Reclaim's witness network is honest, and the backend does not substitute a different grossPay value into the Noir circuit."

For production: the Reclaim proof would be verified on-chain (not just off-chain in the backend), and the grossPay value would be extracted directly from the verified proof parameters rather than being passed as a separate input.

---

13. Testing Checklist

Circuit Tests

- [ ] `nargo prove` succeeds with default Prover.toml values ($10,000 income, $2,500 claim)
- [ ] `nargo verify` returns "Verification successful"
- [ ] Proof fails when `gross_pay_cents = 200000` (below $3,000 threshold)
- [ ] Proof fails when `claim_amount_cents = 600000` (above $5,000 ceiling)
- [ ] Proof fails when `check_date_epoch < max_pay_stub_age_secs` (stale pay stub)

Backend Tests

- [ ] `POST /api/claim` returns 202 with claimId
- [ ] `POST /api/claim` with amount > $5,000 returns 400
- [ ] `GET /api/claim/:id` returns processing state immediately after submit
- [ ] `GET /api/claim/:id` eventually returns submitted/approved after proof generation
- [ ] zkFetch returns `grossPay` in extractedParameters (log and verify)
- [ ] Reclaim `verifyProof` returns `true`

Frontend Tests

- [ ] Form renders, validates input (wallet must start with `xion1`, amount 1–5000)
- [ ] Submit shows ProofProgress with animated spinner on active step
- [ ] Approved state shows green checkmark and dollar amount
- [ ] Transaction hash links to Xion explorer
- [ ] Claim ID lookup works from separate form
- [ ] `/claim/:id` page auto-refreshes every 3 seconds

End-to-End

- [ ] Full flow: form submit → backend processes → Xion tx included in block → frontend shows approved
- [ ] Claim queryable on-chain via `xiond query wasm contract-state smart $CONTRACT '{"get_claim":{"id":"..."}}'`

Appendix: Key External Links

Resource	URL
Reclaim Dev Portal	https://dev.reclaimprotocol.org
Xion Testnet Faucet	https://xion-testnet.faucet.burnt.com
Xion Testnet Explorer	https://explorer.xion-testnet-2.burnt.com
Xion RPC	https://rpc.xion-testnet-2.burnt.com:443

Resource	URL
Nargo Install	https://noir-lang.org/docs/getting_started/installation
CosmWasm Optimizer	https://github.com/CosmWasm/optimizer
zkFetch Docs	https://docs.reclaimprotocol.org/zkfetch/installation
Gusto Demo API	https://docs.gusto.com/embedded-payroll/reference

End of specification. Build order: circuit → contracts → backend → frontend. Each component is independently testable before wiring to the next.